

《跨国聊天系统"Raven Chat"技术复盘报告》



作者：[刘俊言]

天津大学 电气自动化与信息工程学院 通信工程专业

2026 年 5 月

天津大学

目录

第一章 项目背景与需求分析	3
1.1 项目缘起	3
1.2 需求分析	3
1.3 设计约束	4
第二章 技术架构设计	5
2.1 整体架构	5
2.2 技术选型理由	5
2.3 架构演进故事	6
第三章 核心技术实现	7
3.1 FastAPI 后端设计	7
3.2 前端 PWA 设计	11
第四章 运维与部署	16
4.1 设备选型与系统安装	16
4.2 开机自启方案	17
第五章 产品化与分发方案	18
5.1 Windows 安装包制作	18
5.2 用户使用流程	19
第六章 踩坑记录与技术反思	20
6.1 关键踩坑记录	20
6.2 当前方案的局限性	22
第七章 总结与展望	23
7.1 项目成果	23
7.2 个人收获	23
7.3 致谢	24
附录	24
附录 A: 完整源码目录结构	24
附录 B: Nginx 完整配置	25
附录 C: 系统启动脚本	26
附录 D: 项目运行截图	28

摘要:本文介绍了一款面向跨国私人通信场景的轻量级即时聊天系统“Raven Chat”的设计、实现与部署全过程。针对中国与柬埔寨之间常规通信工具存在的网络阻断、注册限制等问题，基于 FastAPI 后端框架与原生 Web 前端技术，开发了支持文字与图片传输的 PWA 聊天应用。系统在零资金投入、无公网 IP 的约束下，采用 Ngrok 内网穿透与 Nginx 反向代理相结合的网络架构，在单条免费隧道上实现了多服务复用的路径分发。部署层面，将服务运行于刷写 Armbian 系统的消费级电视盒子（HK1BOX）上，通过 crontab 开机自启与三重日志抑制机制，保证了 7×24 小时无人值守的稳定运行。经实际使用验证，系统可满足两人间稳定、私密、低延迟的跨国通信需求。本文完整记录了从需求分析、技术选型、架构演进到运维部署的全部工程实践，并总结了踩坑经验与技术反思，可为相似约束条件下的个人化网络服务开发提供参考。

关键词:即时通信；内网穿透；FastAPI；Ngrok；PWA；电视盒子部署

第一章 项目背景与需求分析

1.1 项目缘起

2026 年四月，笔者的一位柬埔寨好友因通信工具阻断，与中国境内的笔者失去了稳定的联系。市面上的主流即时通信软件，如微信需要中国大陆手机号实名注册，且对海外用户支持有限，Instagram 需要内陆翻墙才可使用，且网络不稳定，SnapChat 软件不能永久保存聊天记录，而且软件包含大量与联系无关的冗余功能，均无法满足两人之间简单、稳定、私密的通信需求。

笔者作为天津大学通信工程专业大二学生，在学习了 Python 编程后，决定利用课余时间，设计并实现一套完全自主可控的私人聊天系统。该项目被命名为 “Raven Chat”，取意渡鸦传信。

1.2 需求分析

编号	需求描述	优先级	实现情况
R1	支持文字和图片的双向传输	最高	已实现
R2	7×24 小时在线，无需双方同时在线	最高	已实现
R3	跨国可访问，不受网络阻断影响	最高	已实现
R4	无需注册账号，打开即用	高	已实现
R5	可区分消息发送者身	中	已实现

	份（中国/柬埔寨）		
R6	低成本部署，使用现有设备	中	已实现
R7	PC 端可安装，方便日常使用	低	已实现
R8	无需翻墙等技术手段上网	低	已实现
R9	支持苹果、安卓等系统的安装包	低	未实现

1.3 设计约束

本系统在设计过程中受到以下条件约束：

a.无公网 IP：家庭宽带不提供固定公网 IP 地址，必须通过内网穿透技术实现外网访问。

b.无资金投入：项目全程保持零资金投入，所有服务使用免费方案；Ngrok 采用免费版，设备为闲置电视盒子。

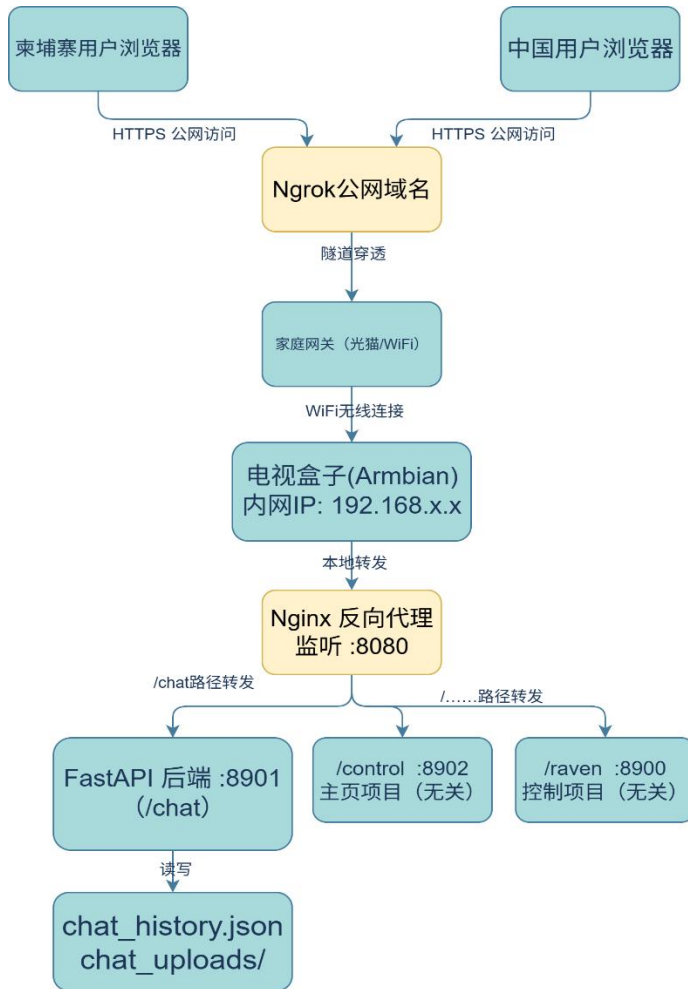
c.无云服务器：未购买或租借任何云服务器；所有计算、存储、网络服务均运行在一台本地设备上。

d.时间约束：从项目构想到首个可用版本交付，总计使用两周课余时间。

e.技术基础：笔者为大二本科生，项目开始前仅具备 Python 编程基础，无全栈开发或运维部署经验。

第二章 技术架构设计

2.1 整体架构



系统支持两条访问路径：①公网路径——柬埔寨用户通过 Ngrok 公网域名经家庭网关转发至电视机盒子；②局域网路径——中国用户在同一校园 WiFi 网络下可直接通过盒子内网 IP（端口 8080）访问。Nginx 根据 URL 路径前缀 /chat 将请求转发到 FastAPI 后端（8901 端口），其他路径 /raven、/control 分发至无关项目端口。FastAPI 服务将聊天记录和图片分别持久化至 JSON 文件和本地文件夹。

2.2 技术选型理由

本系统在技术选型上遵循三项基本原则：①零资金投入，所有组件均使用开源或免费方案；②够用即可，不引入不必要的复杂性；③笔者已掌握或可快速学习的技术优先。

各核心组件的选型理由如下表所示。

组件	选择方案	选型理由
后端框架	FastAPI	基于 Python，笔者已具备该语言基础；支持异步处理，性能满足两人聊天需求；自动生成 API 文档，便于调试；部署命令仅需一行 <code>uvicorn.run()</code>
前端技术	HTML + CSS + JavaScript	无需学习 React/Vue 等框架，降低开发时间成本；通过 PWA 的 <code>manifest.json</code> 可实现“添加到桌面”的近原生体验
数据存储	JSON 文件	两人聊天场景下数据量极小（预估每日不超过百条），无需引入 MySQL/PostgreSQL；JSON 可手动查看和备份，调试友好
内网穿透	Ngrok 免费版	无需自建公网服务器或申请公网 IP；支持 HTTPS，一键启动；免费套餐月连接数足以支撑私人通信
Web 服务器	Nginx	轻量高性能，适合电视盒子这类资源受限设备；支持基于 URL 路径的反向代理，实现单端口多服务复用
部署设备	电视盒子 + Armbian	硬件成本为零；功耗仅约 5W，适合 7×24 小时运行；Armbian 提供完整的 Linux 环境，与服务器部署无异
开机自启	crontab @reboot	无需编写 systemd 服务文件，配置简单；结合 Shell 脚本实现全自动启动链

表中所列均为目前业界广泛应用的开源技术，社区生态成熟，遇到问题时便于查找解决方案。

本系统的技术路线可概括为：以 Python 为后端核心，以 Nginx 为流量调度中心，以 Ngrok 架设公网桥梁，以 JSON 文件轻量化存储，以 PWA 实现跨平台客户端。这一组合在保证功能完整性的前提下，将全系统的复杂度与资源占用控制在最低水平。

2.3 架构演进故事

本系统的部署方案并非一开始就定型，而是经历了一次硬件迁移。

第一阶段：游戏本部署（已废弃）

项目初期，服务运行于笔者日常使用的游戏笔记本上。笔记本安装 Windows 系统，通过电源适配器保持供电，开启电池养护模式以防止过充。Ngrok 隧道、nginx 代理和 FastAPI 服务以后台进程方式运行。

该方案运行一段时间后暴露出严重问题：电脑作为开发机不适宜长时间开机运行，且三大部分内容长时间在后台进程运行，占用电脑内存，暴露出“依赖个人主力机作为服务器”的脆弱性。

第二阶段：电视盒子部署（当前方案）

吸取教训后，笔者决定将服务迁移至一台闲置的电视盒子。该盒子搭载 Amlogic S905X3 芯片，刷入 Armbian 系统（基于 Debian 的 ARM 版本），通过 WiFi 连接学校网关。

迁移后的优势如下：

- a.低功耗：**盒子功耗约 5W，仅为笔记本的 1/10 以下，长期运行电费可忽略不计；
- b.高稳定性：**盒子专用于服务，无其他软件干扰，配合 crontab 开机自启，断电恢复后无需人工干预；
- c.物理隔离：**服务运行于独立设备，不影响个人电脑的正常使用和关机。

演进启示

此次迁移让笔者认识到，长期运行的网络服务必须部署在专有设备上。个人电脑适用于开发测试阶段，而生产环境需要独立硬件。电视盒子作为低成本 Linux 边缘计算节点，对于个人小型服务而言是理想选择。

第三章 核心技术实现

3.1 FastAPI 后端设计

3.1.1 路由设计

FastAPI 通过装饰器将 URL 路径与处理函数绑定。本系统共设计 6 个路由端点，覆盖页面渲染、消息收发、文件访问、应用下载和 PWA 配置五大功能域。

方法	路径	功能	返回类型
GET	/chat	返回完整聊天页面	HTML
GET	/chat/history	返回全部聊天记录	JSON
POST	/chat/send-combined	接收文字与图片消息	JSON
GET	/chat/uploads/{filename}	访问已上传的图片	文件
GET	/chat/download/pc	下载 Windows 安装包	文件
GET	/chat/manifest.json	提供 PWA 应用清单	JSON

路由设计遵循 RESTful 风格，每个路径对应单一职责。其中 /chat/send-combined 是核心端点，同时处理文字和图片的上传，合并为一个 POST 请求以减少网络往返次数。

所有路由统一挂载在 /chat 路径前缀下，与 Nginx 反向代理的路径匹配规则精确对应。当 Nginx 收到 /chat/xxx 请求时，将请求原文转发至 FastAPI 的 8901 端口，由框架内部路由表完成二级分发。

3.1.2 消息模型

```
{
  "time": "05-03 15:30",
  "content": "Hello!",
  "image": "20260503_153000.jpg",
  "me": true
}
```

字段	类型	说明
time	string	消息发送时间，格式 MM-DD HH:MM，由服务端时间戳生成
content	string	文字消息正文；若仅发送图片则为空字符串
image	string / null	图片文件名（存储于 chat_uploads/ 目录）；无图片时为 null
me	boolean	true 表示发送者为"中国用户"，false 表示"柬埔寨用户"

设计考量：

a.时间由服务端生成：避免客户端时钟不一致导致消息顺序错乱；

b.图片与文字共用一条消息：用户可同时发送文字和图片，前端渲染时图片在文字上方显示；

c.身份字段用布尔值：仅区分"我/对方"，符合两人聊天场景，无需复杂用户体系。

所有消息按时间顺序追加存储在 chat_history.json 文件中，前端每次轮询获取全量列表后整体渲染。

3.1.3 三重日志抑制机制

Uvicorn 作为 ASGI 服务器，默认输出大量访问日志和错误日志。在普通服务器上这些日志有助于监控和排错，但在电视盒子这类使用 SD 卡或 eMMC 闪存作为存储介质的嵌入式设备上，无意义的日志持续写入会带来两个问题：①占用 I/O 带宽，影响服务响应速度；②频繁写入加速 flash 存储磨损，缩短设备寿命。

为此，本系统设计了三层递进的日志抑制策略，确保盒子长期运行时存储写入量

降至最低。

第一层：代码顶部禁用 Logger

在 import 完成后、任何业务逻辑执行前，立即禁用 Uvicorn 相关的全部 Python Logger：

```
logging.getLogger("uvicorn").disabled = True

logging.getLogger("uvicorn.access").disabled = True

logging.getLogger("uvicorn.error").disabled = True

logging.getLogger("uvicorn.asgi").disabled = True
```

此层从根源上阻止 Uvicorn 内部日志对象产生输出。logging.basicConfig 将根日志级别设为 CRITICAL，只允许致命错误输出（实际运行中几乎不会触发）。

第二层：启动参数关闭访问日志

在 uvicorn.run() 中设置 access_log=False：

```
uvicorn.run(app, host="0.0.0.0", port=8901, access_log=False, ...)
```

此参数直接关闭 Uvicorn 内置的 HTTP 访问日志（每条请求一行），是官方提供的日志控制入口。

第三层：空日志配置字典

在 uvicorn.run() 的 log_config 参数中传入一个最小化的空配置：

```
log_config={

    "version": 1,

    "disable_existing_loggers": True,

    "handlers": {},

    "loggers": {}

}
```

handlers 和 loggers 均为空字典，意味着没有任何日志处理器被注册。即使前两层有遗漏的日志输出，也会因无处理器可接收而被丢弃。

三层防御逻辑：第一层阻止 Logger 产生日志；第二层阻止 HTTP 访问日志；第三层从配置层面清空所有日志处理器。三层叠加保证了无论 Uvicorn 内部经过哪条路径输出日志，最终都不会写入磁盘。

实测效果：部署前，盒子每日产生约 2–5 MB 的日志文件；部署三重抑制后，

chat.log 控制在每日数百字节以内（仅包含 Python 致命错误，运行至今未触发）。

3.1.4 图片上传与存储

图片传输是与文字并列的核心需求。系统使用 HTTP 的 multipart/form-data 格式在同一个 POST 请求中承载文字和图片。

上传流程：

1. 前端将文字 (text) 和图片文件 (image) 封装为 FormData 对象
2. 通过 POST /chat/send-combined 发送至后端
3. 后端从 request.form() 中分别提取文字和图片字段
4. 若图片字段存在且包含文件名，提取原始扩展名，结合当前时间戳生成唯一文件名
5. 将图片内容写入 chat_uploads/ 目录
6. 在消息 JSON 中记录图片文件名 (image 字段)

文件名生成逻辑：

```
ext = image_file.filename.split(".")[-1] if "." in image_file.filename else "jpg"
fn = f"{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}.{ext}"
```

文件名格式为 YYYYMMDD_HHMMSS.扩展名，精确到秒，避免重名冲突。

图片访问：存储后的图片通过 GET /chat/uploads/{filename} 路由返回。该路由直接使用 FastAPI 的 FileResponse，从 chat_uploads/ 目录读取对应文件并返回二进制流。

存储现状：上线至今，chat_uploads/ 目录累计存储图片若干张，总占用空间约 668kB。

潜在问题与改进方向：当前无图片压缩处理，用户上传原图可能导致存储快速增长。后续可引入 Pillow 库在服务端对图片进行尺寸限制和格式优化。

3.2 前端 PWA 设计

3.2.1 短轮询消息同步

本系统采用 HTTP 短轮询实现消息同步，这是在不引入 WebSocket 等长连接技术下最简洁的实时通信方案。

核心机制：前端通过 JavaScript 的 `setInterval` 以 3 秒为周期向 `GET /chat/history` 端点请求全量消息列表，收到响应后重新渲染整个聊天界面。

```
async function load(){

  let res = await fetch('/chat/history');

  let data = await res.json();

  let box = document.getElementById('chat-container');

  // 条件 1: 判断用户是否在聊天底部

  const isAtBottom = box.scrollTop + box.clientHeight >= box.scrollHeight - 10;

  // 条件 2: 判断是否有新消息到达

  const isNewMessage = data.length > lastCount;

  lastCount = data.length;

  // 清空并重新渲染所有消息

  box.innerHTML = '';

  data.forEach(m => {

    let div = document.createElement('div');

    div.className = 'msg ' + (m.me ? 'me' : 'other');

    let content = '';

    if (m.image) content += ``;

    div.innerHTML = content + m.content + `<div class="time">${m.time}</div>`;

    box.appendChild(div);

  });

  // 两个条件同时满足才自动滚动到底部

  if (isAtBottom && isNewMessage) {

    box.scrollTop = box.scrollHeight;
```

```

    }
}

setInterval(load, 3000);

```

滚动触发条件——“双绿灯”逻辑：

自动滚动到底部的触发必须同时满足两个条件：

条件	判断方式	含义
<code>isAtBottom</code>	<code>scrollTop + clientHeight >= scrollHeight - 10</code>	用户当前位于聊天底部 (10px 容差)
<code>isNewMessage</code>	<code>data.length > lastCount</code>	服务器有新消息到达 (消息总数增加)

两个条件缺一不可。不同场景下的行为如下：

场景	<code>isAtBottom</code>	<code>isNewMessage</code>	是否滚动	用户体验
底部等待，新消息到达	1	1	自动滚动	正常的实时对话体验
翻看历史记录，新消息到达	0	1	不滚动	不打断历史浏览
在底部等待，无新消息	1	0	不滚动	保持当前位置
看历史记录，无新消息	0	0	不滚动	保持当前位置

传统的短轮询方案通常仅判断新消息数，有新消息即强制滚动，导致用户在翻阅历史记录时频繁被“拖回”底部。本系统引入双条件判断后，解决了这一体验问题：只有用户主动停留在底部时，才自动展示最新消息；用户向上翻阅历史，系统尊重用户浏览意图，不打扰。

全量渲染策略：每次轮询收到消息列表后，前端清空 chat-container 的 innerHTML 并遍历所有消息重新创建 DOM 节点。该方式实现简单，在消息总量较少的两人聊天场景下（预估全生命周期不超过数万条），DOM 重建耗时低于浏览器一帧（16ms），不会造成可感知的卡顿。若未来消息量增长，可优化为增量追加 DOM 节点。

方案评价：

优点：实现极简，无需 WebSocket 等长连接协议支持；与 RESTful API 设计天然

兼容；双条件滚动逻辑保护用户浏览上下文。

局限：消息到达延迟最高 3 秒；每次请求传输全量数据，存在一定带宽冗余。

适用性论证：对于两人、低频、跨国的私人聊天场景，3 秒延迟在日常对话中几乎不可感知；全量数据在数百至数千条消息量级下仅数十 KB，对带宽影响可忽略。该方案在目标场景中是合理的工程权衡。

3.2.2 PWA 可安装能力

为使 Web 聊天页面获得接近原生应用的体验，系统配置了 PWA（Progressive Web Application）所必需的 Web App Manifest 文件。

manifest.json 内容：

```
{
  "name": "Raven Chat",
  "short_name": "Raven",
  "start_url": "/chat",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#007bff",
  "icons": [
    {
      "src": "/static/raven-icon.png",
      "sizes": "1024x1024",
      "type": "image/png"
    }
  ]
}
```

各字段效果说明：

字段	作用
name / short_name	添加到桌面时显示的应用名称

start_url	启动时打开的页面路径
display: "standalone"	隐藏浏览器地址栏和菜单，以独立窗口运行
background_color	启动闪屏的背景色
theme_color	状态栏/标题栏颜色
icons	桌面图标，建议 1024×1024 的 PNG

用户操作流程：

1. 在手机浏览器中打开聊天链接
2. 浏览器检测到 manifest.json 后自动弹出"添加到主屏幕"提示（或手动从菜单选择）
3. 添加后桌面出现 Raven Chat 图标，点击即以全屏独立应用方式启动

PWA 方案无需经过应用商店审核，无需为 iOS 和 Android 分别开发，一次部署多端可用。这是本系统在"零资金、快速交付"约束下替代原生 App 安装包的较为合适的方案。

3.3 Nginx 反向代理配置

3.3 Nginx 反向代理配置

Nginx 在本系统中的角色是"流量调度中心"。所有外部请求首先到达电视盒子的 8080 端口（由 Nginx 监听），Nginx 根据 URL 路径前缀将请求转发至对应的后端服务。

部分配置：

```
# 聊天项目

location /chat {

    proxy_pass http://127.0.0.1:8901;

    proxy_set_header Host $host;

    proxy_set_header X-Real-IP $remote_addr;

    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

    proxy_set_header X-Forwarded-Proto $scheme;

    proxy_redirect off;

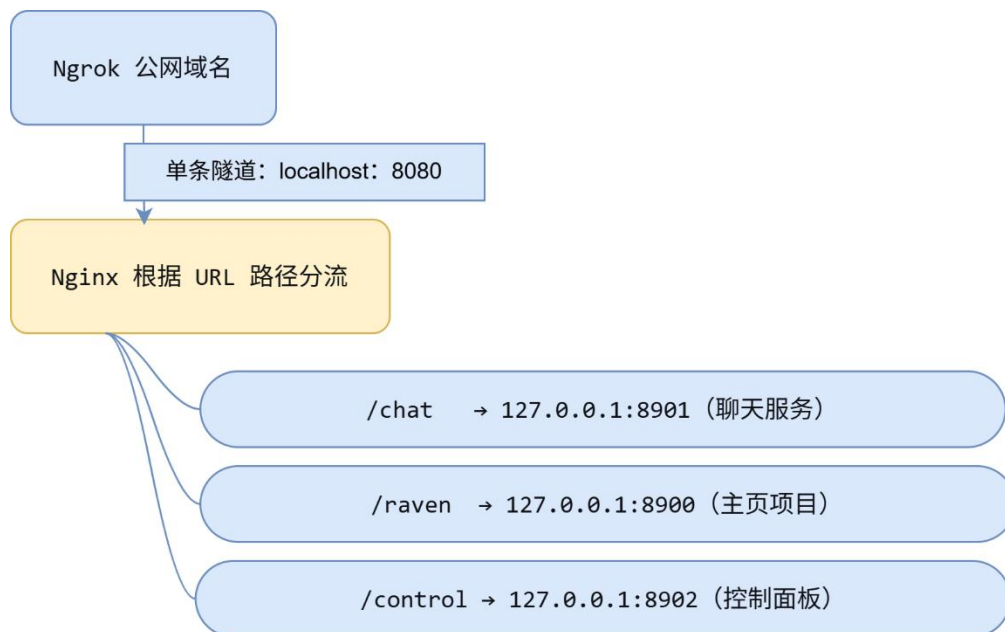
}
```

核心配置解析：

指令	作用
<code>listen 8080</code>	Nginx 监听 8080 端口，Ngrok 隧道将公网流量转发至此
<code>location = /</code>	精确匹配根路径 /，返回 200 OK，用于 Ngrok 隧道健康检查
<code>location /chat</code>	前缀匹配，所有 /chat/* 请求被转发至 FastAPI（8901 端口）
<code>proxy_pass</code>	将匹配的请求原样转发至指定后端地址
<code>proxy_set_header</code>	向后端传递原始请求信息（Host、真实 IP、协议），便于日志记录和后续扩展

单隧道多服务复用的核心思路：

Ngrok 免费版仅提供一条 TCP 隧道。若每个项目独立占用一条隧道，免费额度将无法满足。通过 Nginx 的 `location` 指令，利用 URL 路径前缀作为"虚拟隧道标识"，在单条隧道上实现了流量分发：



该方案不仅节约了隧道资源，还使得后续新增项目无需额外隧道，只需在 Nginx 中新增一个 `location` 块即可。

第四章 运维与部署

4.1 设备选型与系统安装

硬件选型:本系统最终部署在一台闲置的 HK1BOX 电视盒子上，核心配置如下：

项目	参数
型号	HK1BOX
芯片型号	Amlogic S905X3
CPU 架构	ARM Cortex-A55（四核）
内存	4 GB DDR3
存储	64 GB eMMC
网络	双频 WiFi（2.4G/5G）+ 千兆以太网口
原始系统	Android 9.0
当前系统	Armbian（基于 Debian）
功耗	约 5 W

选择电视盒子的原因有三：①闲置设备，硬件成本为零；②ARM 架构功耗极低，适合 24 小时不间断运行；③可刷入 Armbian 系统，获得完整的 Linux 环境，4 GB 内存为多服务运行提供充足余量。

系统安装

Armbian 是专为 ARM 开发板/电视盒子定制的 Debian 发行版。安装步骤如下：

1. 从 Armbian 官方仓库下载适配 Amlogic S905X3 芯片的系统镜像
2. 使用 balenaEtcher 工具将镜像写入 SD 卡
3. 将 SD 卡插入电视盒子并通电启动
4. 首次启动完成后，通过 SSH 登录系统
5. 执行 `armbian-config` 将系统从 SD 卡迁移至 eMMC 存储
6. 配置 WiFi 连接家庭网络（亦可使用千兆网口），获得固定内网 IP

系统安装完成后，盒子成为一台标准的 Linux 服务器，支持 Python、Nginx、Ngrok 等全部依赖组件的安装与运行。

项目目录结构

在 /root/myprojects/chat/ 目录下，聊天项目的完整文件组织如下：

```

/root/myprojects/chat/

├── main.py           # FastAPI 主程序
├── chat_history.json  # 聊天记录持久化文件
├── chat_uploads/     # 上传图片存储目录
├── Ravenchat_Setup.exe # Windows 安装包
├── start_chat.sh      # 开机自启脚本
├── static/           # 静态资源（图标等）
│   └── raven-icon.png
└── venv/             # Python 虚拟环境
  
```

4.2 开机自启方案

长期运行的服务器必须能够在断电重连后自动恢复全部服务，避免人工干预。本系统使用 crontab @reboot 配合 Shell 脚本实现全自动启动链。

crontab 配置：

```
@reboot /root/myprojects/chat/start_chat.sh
```

@reboot 指令在系统每次启动时执行指定脚本，比手动编写 systemd 服务更简洁直观

启动脚本 start_chat.sh：

详见附录

启动链分析：

步骤	操作	延时	原因
1	sleep 15	15 秒	等待系统完全启动、WiFi 连接建立
2	nginx	—	先启动反向代理，承接后续流量
3	nohup python3	—	后台启动聊天服务，输出重定向到日志

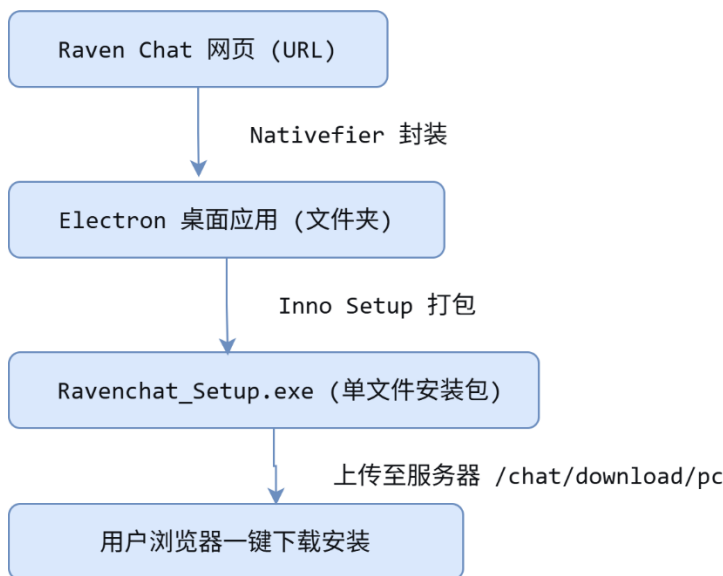
	main.py &		
4	sleep 5	5 秒	确认后端服务就绪后再暴露公网入口
5	nohup ngrok http 8080 &	—	最后启动隧道，确保公网请求到达时后端已就绪

nohup 确保脚本退出后服务不被终止；& 将进程置于后台运行；> xxx.log
2>&1 将标准输出和标准错误均重定向到日志文件，配合三重日志抑制机制使系统输出降至最低。

第五章 产品化与分发方案

5.1 Windows 安装包制作

柬埔寨好友日常使用 Windows 笔记本电脑，为方便其访问聊天系统，笔者将 Raven Chat 网页封装为独立的 Windows 桌面应用程序，并制作了标准安装包。



第一步：Nativefier 封装网页为桌面应用

Nativefier 是一个开源命令行工具，可将任意网页封装为基于 Electron 的独立桌面应用。执行命令：

```

nativefier --name "Raven Chat" \

    --platform windows \

    --arch x64 \

    --icon raven-icon.png \
  
```

```
https://xxxxxxxxxxxxxxxxxxxxx
```

该命令生成一个包含完整 Electron 运行时的文件夹，双击其中的 Raven Chat.exe 即可以独立窗口打开聊天页面，体验与原生 Windows 应用无异。

Nativefier 的核心价值在于：无需学习 Electron 开发框架，无需编写任何 C++ 或 Node.js 代码，一条命令即可将任意 Web 应用转化为桌面应用。

第二步：Inno Setup 打包为标准安装程序

Nativefier 生成的文件夹需手动复制到目标电脑，分发不便。为提供标准 Windows 安装体验，笔者使用 Inno Setup 将文件夹打包为单个 .exe 安装包。

Inno Setup 是一款免费开源的 Windows 安装包制作工具，通过编写 .iss 脚本定义安装行为——指定源文件夹路径、目标默认安装路径（如 C:\Program Files\Raven Chat）、开始菜单快捷方式、桌面快捷方式以及卸载程序。

编译脚本后生成 Ravenchat_Setup.exe，用户双击即可启动标准安装向导：选择安装路径 → 确认许可 → 完成安装。

第三步：集成至下载路由

将生成的安装包文件放置于项目根目录，FastAPI 通过 GET /chat/download/pc 路由直接返回文件：

```
@app.get("/chat/download/pc")

async def download_pc():

    return FileResponse(SETUP_EXE, filename="Ravenchat_Setup.exe")
```

用户在聊天页面点击左上角菜单中的 "Download PC Version" 即可一键下载安装包。

分发效果

柬埔寨好友通过该安装包在其 Windows 笔记本上安装了 Raven Chat，此后无需每次打开浏览器输入链接，点击桌面图标即进入聊天界面。这是本系统从"能用"走向"好用"的关键一步。

5.2 用户使用流程

Raven Chat 面向两位用户——中国用户与柬埔寨用户——提供统一的聊天入口与差异化访问路径。

首次访问

1. 柬埔寨用户在浏览器输入 Ngrok 公网链接 (<https://staining-pawing-survivor.ngrok-free.dev/chat>)；中国用户可同样使用公网链接，亦可在家庭网络内直接访问盒子内网 IP (<http://盒子 IP:8080/chat>)
2. 页面加载后显示身份选择界面——"I am in China" / "I am in Cambodia"
3. 用户根据自身所在地点点击对应按钮，系统记录其身份 (me = true/false)
4. 身份确认后进入聊天主界面

日常聊天

1. 在底部输入框键入文字，或点击"Select Image"选择图片
2. 点击"Send Message"按钮发送
3. 发送期间按钮显示"Sending..."，状态栏显示"Sending message..."
4. 发送成功后状态栏提示"Sent successfully!"，1.5 秒后自动消失
5. 对方在 3 秒内（轮询周期）自动收到新消息

PC 端安装（可选）

1. 点击左上角"..."菜单按钮
2. 选择"Download PC Version"
3. 下载 Ravenchat_Setup.exe 并运行安装向导
4. 安装完成后桌面出现 Raven Chat 图标，双击即可启动

移动端使用

1. 在手机浏览器打开聊天链接
2. 浏览器检测到 PWA manifest 后弹出"添加到主屏幕"提示
3. 添加后手机桌面出现 Raven Chat 图标，点击以全屏模式运行

第六章 踩坑记录与技术反思

6.1 关键踩坑记录

独立开发与部署过程中，所踩技术陷阱多于代码逻辑本身的问题。这些经历是项目中收获最大的部分。

序号	踩坑	原因	解决方案
----	----	----	------

1	欲以 IP 地址自动判断用户国籍，结果不准确	免费 IP 数据库质量欠佳，且 Ngrok 代理后服务端获取的是 Ngrok 节点 IP	改为用户启动时手动选择身份"I am in China / I am in Cambodia"
2	笔记本部署不稳定，断电即服务中断	个人电脑不适合长时间运行；电池鼓包、系统更新重启等意外频繁	迁移至 HK1BOX 电视盒子，刷 Armbian 系统，功耗仅 5W，专机专用
3	Ngrok 免费隧道仅一条，多个项目需同时对外服务	免费套餐限制一条 TCP 隧道	部署 Nginx 反向代理，按 URL 路径前缀 (/chat、/raven、/control) 分发至不同后端端口
4	Uvicorn 日志持续写入，消耗存储 I/O 并加速闪存磨损	Uvicorn 默认输出大量 HTTP 访问日志	设计三层递进的日志抑制策略（禁用 Logger + 关闭 access_log + 空 log_config）
5	Nginx 配置多次修改均不生效，卡住数小时	开发测试阶段关闭脚本只终止了 Ngrok，Nginx 进程未杀；重复启停导致累积 40 余个 nginx.exe 进程，首个主进程持有原始配置	强制终止全部 nginx 进程后重新启动单实例；此后改用 nginx -s reload 热重载配置

案例详述：Nginx 后台进程残留与配置失效

该问题是本项目中排查时间最长的一例。

开发测试阶段，为操作便捷编写了一键启停脚本。启动脚本同时拉起 Nginx 和 Ngrok；停止脚本却仅终止 Ngrok 进程。每次重启流程新增 2-4 个 Nginx 进程，而旧进程从未被清理。多次迭代后，系统中累积了 40 余个 nginx.exe 进程。

现象：修改 Nginx 配置文件 nginx.conf 后重启脚本，配置始终不生效。无论改动路径规则还是端口设置，浏览器访问结果均与修改前一致。反复检查语法无误，甚至删除配置文件后服务器仍在运行。

排查过程：该问题困扰笔者数小时。最终打开 Windows 任务管理器，发现进程列表中有 40 余个 nginx.exe 进程。每次运行启动脚本都会新增 2-4 个 Nginx 进程，而停止脚本只终止了 Ngrok，Nginx 进程持续累积。

这 40 个进程中，第一个启动的作为主进程持有原始配置，后续新增的进程都服从主进程的配置。所有对配置文件的修改从未被任何一个进程加载——主进程永远是最早那个，新进程只是它的子进程，配置继承自主进程的内存，不重新读取磁盘文件。

解决：手动强制终止所有 nginx.exe 进程，确保进程列表为空后，重新启动单个 Nginx 实例。此后修改配置使用 `nginx -s reload` 命令热重载，不再反复启动新进程。

Raven Chat 作为独立项目已完成从需求分析、架构设计、编码实现到部署运维的完整闭环。其技术价值在于：

验证了一条可行的技术路线：在无公网 IP、零资金投入的约束下，FastAPI + Ngrok + Nginx + PWA 的组合能够支撑跨国私人通信的完整链路。该结论并非来自理论推演，而是经过实际部署与长期运行检验的工程结果。

积累了可迁移的工程经验：三重日志抑制策略可应用于任何部署在嵌入式设备上的 Python 服务；单隧道路径复用的 Nginx 配置模式可直接用于后续项目的对外暴露；Nativefier + Inno Setup 的 Web 应用打包流程为快速分发提供了可复用的操作模板。

为更大规模的自研系统提供了技术基线：本项目所积累的 FastAPI 后端开发、Nginx 反向代理配置、Armbian 系统运维、内网穿透方案选择等能力，可直接支撑更复杂的网络系统开发（如笔者后续规划的自研网络控制系统——Raven 星枢，该项目的设计蓝图已独立成文，不在本报告范围内展开）。

6.2 当前方案的局限性

如实列出本系统目前存在的不足与技术债务：

1. **短轮询效率低：**3 秒间隔带来不必要的 HTTP 请求，消息延迟最高达 3 秒。两人聊天场景下可接受，但方案本身不具备扩展到多用户或高频通信场景的能力。
2. **无身份认证：**当前任何人知道链接即可访问聊天页面并读取历史记录。虽为两人私人通信设计，安全等级较低。
3. **消息明文存储：**chat_history.json 中所有消息以明文 JSON 存储，服务器端可查看全部聊天内容。
4. **中心化架构：**所有消息经电视盒子中转，盒子断电则服务全断；且消息在服务端以明文存在，理论上服务端可被查看。

5. **无移动端原生安装包**：虽通过 PWA 可添加至手机桌面，但未提供 APK/IPA 格式的安装包。
6. **图片无压缩**：原图直接上传存储，长期可能占满盒子空间。

第七章 总结与展望

7.1 项目成果

本项目从真实需求出发，完成了一款轻量级跨国私人聊天系统的完整设计与交付。主要成果如下：

a.核心系统实现：基于 FastAPI 开发后端服务，包含 6 个路由端点，覆盖文字/图片消息收发、历史记录拉取、文件访问与应用下载等完整功能；前端采用原生 HTML/CSS/JavaScript 构建 PWA，通过短轮询机制实现消息同步，并配置 Web App Manifest 支持移动端添加到桌面。

b.网络架构设计：在无公网 IP、零资金投入约束下，使用 Ngrok 免费隧道实现公网穿透，通过 Nginx 反向代理以 URL 路径前缀在单条隧道上复用多个后端服务，解决了免费隧道数量限制问题。

c.部署与运维：将服务部署于刷写 Armbian 系统的 HK1BOX 电视盒子上，利用 crontab @reboot 与 Shell 启动脚本实现全自动开机自启与 Ngrok 隧道保活，达成 7×24 小时无人值守运行。

d.产品化分发：使用 Nativefier 将网页封装为 Windows 桌面应用，通过 Inno Setup 打包为标准安装包，降低了非技术用户的使用门槛。

e.工程优化：针对嵌入式设备存储寿命问题，设计了三重日志抑制机制，将日均日志输出从数 MB 降至数百字节以内。

7.2 个人收获

通过本项目，笔者完整经历了一个系统从构想到落地的全过程，在以下方面获得显著提升：

a.全栈开发能力：从仅掌握 Python 基础，到能够独立完成 FastAPI 后端、原生前端页面及 Nginx 配置的全链路开发。

b.网络工程理解：实践了内网穿透、反向代理、路径路由、静态文件服务等网络服务部署的关键技术，对“服务如何从内网到达公网”有了深刻的直观认识。

c.运维与排错经验：面对笔记本断电、Nginx 多进程残留、Ngrok 隧道路径失效等问题，学会了系统性地定位根因并根治，理解了“开发环境与生产环境的差异”。

d.产品化思维：意识到“用起来”比“写出来”更需要额外的工作，从命令行启动到一键安装包，是从开发者视角到用户视角的重要跨越。

e.项目闭环能力：独立完成了需求分析、技术选型、架构设计、编码实现、部署交付、文档撰写六个环节，建立了项目驱动的自学与问题解决模式。

7.3 致谢

感谢柬埔寨好友作为本项目的真实用户与长期测试者。其持续的使用反馈是系统迭代完善的主要推动力。

感谢在项目过程中所有提供间接帮助的开源社区（FastAPI、Uvicorn、Nginx、Ngrok、Nativefier、Inno Setup、Armbian、Tailscale），以及各类技术文档的撰写者。站在这些优秀基础设施之上，个人开发者才能以极低成本完成完整系统的构建。

感谢天津大学提供的学习环境 with 网络条件，使得本项目能够在校园宿舍内完成硬件部署与远程运维。

附录

附录 A：完整源码目录结构

```
/root/myprojects/chat/

├── main.py                # FastAPI 主程序（全部后端逻辑）
├── chat_history.json      # 聊天记录持久化文件
├── chat_uploads/         # 上传图片存储目录
├── Ravenchat_Setup.exe    # Windows 桌面安装包
├── start_chat.sh         # 开机自启 Shell 脚本
├── static/               # 静态资源目录
│   └── raven-icon.png    # PWA 桌面图标
└── venv/                 # Python 虚拟环境
```


附录 B: Nginx 完整配置

```
events {  
  
    worker_connections 1024;  
  
}  
  
http {  
  
    include mime.types;  
  
    default_type text/html;  
  
  
    sendfile on;  
  
    keepalive_timeout 65;  
  
    server {  
  
        listen 8080;  
  
        server_name localhost;  
  
        charset utf-8;  
  
        # 根路径: 健康检查, 用于 Ngrok 隧道验证  
  
        location = / {  
  
            default_type text/plain;  
  
            return 200 "OK";  
  
        }  
  
        # 聊天项目  
  
        location /chat {  
  
            proxy_pass http://127.0.0.1:8901;  
  
            proxy_set_header Host $host;  
  
            proxy_set_header X-Real-IP $remote_addr;  
  
            proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```

        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_redirect off;
    }

    # 主页项目

    location /raven {

        proxy_pass http://127.0.0.1:8900;

        proxy_set_header Host $host;

        proxy_set_header X-Real-IP $remote_addr;

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_redirect off;
    }

    # 控制项目

    location /control {

        proxy_pass http://127.0.0.1:8902;

        proxy_set_header Host $host;

        proxy_set_header X-Real-IP $remote_addr;

        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;

        proxy_set_header X-Forwarded-Proto $scheme;

        proxy_redirect off;
    }
}

```

附录 C：系统启动脚本

```

#!/bin/bash

# 开机自启脚本：Nginx 反向代理 + 聊天服务 + ngrok 公网穿透

```

```
# 等待系统网络就绪
```

```
sleep 15
```

```
# 启动 Nginx 反向代理
```

```
nginx
```

```
# 进入聊天项目目录，后台启动 FastAPI 服务
```

```
cd /root/myprojects/chat
```

```
nohup /root/myprojects/chat/venv/bin/python3 /root/myprojects/chat/main.py > /root/chat.log 2>&1 &
```

```
sleep 2
```

```
# 启动 Raven 主页项目
```

```
cd /root/myprojects/ravenhome
```

```
nohup /root/myprojects/ravenhome/venv/bin/python3 /root/myprojects/ravenhome/main.py > /root/ravenhome.log  
2>&1 &
```

```
# 等待所有后端服务就绪
```

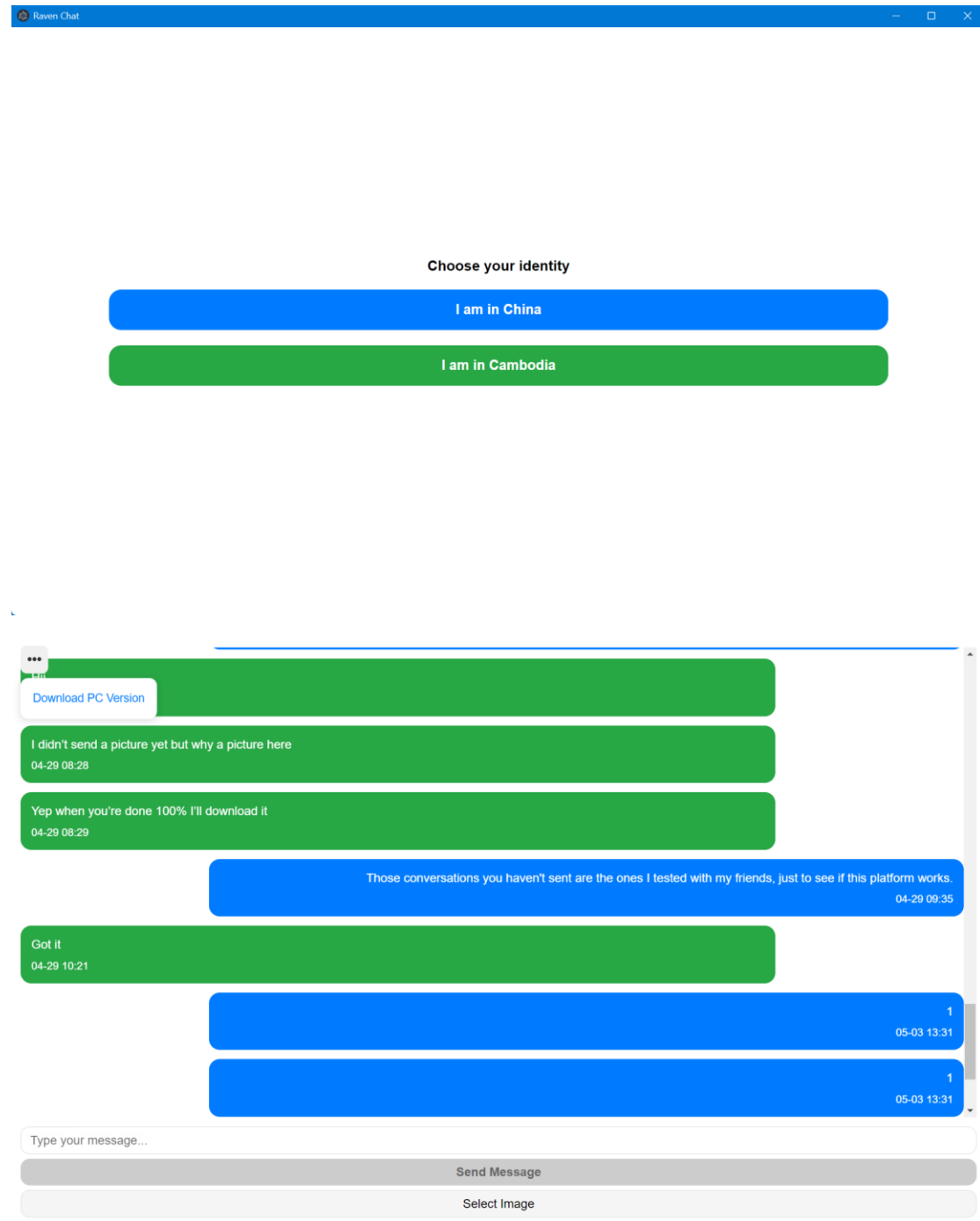
```
sleep 5
```

```
# 启动 ngrok，将本地 8080 端口映射到公网
```

```
nohup ngrok http 8080 > /root/ngrok.log 2>&1 &
```

```
echo "所有服务启动完成！"
```

附录 D：项目运行截图



```

root@armbian:~/myprojects/chat# ps aux | grep -E "python|nginx|ngrok"
root      1430  0.0  0.6 110532 21864 ?        Ssl  14:01   0:00 /usr/bin/python
3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
root      1576  0.0  0.0  10412   960 ?        Ss   14:02   0:00 nginx: master p
rocess /usr/sbin/nginx -g daemon on; master_process on;
nobody    1577  0.0  0.1  10892  3772 ?        S    14:02   0:02 nginx: worker p
rocess
root      1615  0.3  1.2 204248 44140 ?        Sl   14:02   0:59 /root/myproject
s/chat/venv/bin/python3 /root/myprojects/chat/main.py
root      1617  0.2  1.2 127412 41732 ?        Sl   14:02   0:50 /root/myproject
s/ravenhome/venv/bin/python3 /root/myprojects/ravenhome/main.py
root      1684  0.3  1.3 1290256 45568 ?        Sl   14:02   1:04 ngrok http 8080
root      2924  0.0  0.0   6272   2016 pts/0    S+   19:24   0:00 grep --color=au
to -E python|nginx|ngrok
root@armbian:~/myprojects/chat# |

```